Relatório do Projeto — Controle Acadêmico

Parte de pagamentos

Aluno: Juan Henrique Fernandes Rodrigues

Disciplinas: Engenharia de Software, Banco de Dados II e Back-end

Datas usadas neste relatório: fictícias

Sumário

1 Sobre o projeto 1
2 Divisão do trabalho 1
2.1 Pessoa 1 — Ana Clara 1
2.2 Pessoa 2 — Luiz 1
2.3 Pessoa 3 — Juan Henrique 1
3 O que eu fiz 2
4 Procedure e trigger 2
5 Etapas e datas 2
6 Rotas de pagamentos 3
6.1 Rotas web 3
6.2 Rotas da API 3
7 Testes 3
8 Arquivos principais 4
9 Finalização 4

1 Sobre o projeto

Eu usei como base uma aplicação Laravel chamada Controle Acadêmico. O sistema cadastra alunos, cursos e matrículas. Para a minha parte, acrescentei o cadastro de pagamentos dos alunos.

O projeto usa Laravel, PHP, PostgreSQL, Blade, Tailwind, Vite e Laravel Breeze. A ideia é separar cada parte do sistema em model, controller, service, request, view e rota.

No pagamento eu usei a tabela `pagamento`. Cada registro aponta para um aluno pela coluna `student_id`. Também deixei os campos de plano, status, data, forma de pagamento e ativo.

2 Divisão do trabalho

2.1 Pessoa 1 — Ana Clara

A divisão combinada para a Pessoa 1 foi:

- um índice;
- uma function;
- uma trigger;
- backup e restauração com evidências;
- CRUD web e API de clientes;
- `ClienteService`;
- um teste web;
- configuração do repositório, Neon e README;
- rotas de clientes.

2.2 Pessoa 2 — Luiz

A divisão combinada para a Pessoa 2 foi:

- um índice;
- uma function;
- uma procedure;
- CRUD web e API de matrículas;
- `MatriculaService`;
- um teste de API;
- junção dos scripts SQL;
- rotas de matrículas.

2.3 Pessoa 3 — Juan Henrique

Minha parte ficou assim:

- uma procedure;
- uma trigger;
- CRUD web e API de pagamentos;
- `PagamentoService`;
- um teste web;
- um teste de API;
- relatório em PDF;

- rotas de pagamentos.

3 O que eu fiz

Eu criei a migration da tabela `pagamento`, o model `Pagamento`, o `PagamentoRequest` e a factory de pagamento. Também coloquei no model `Student` o relacionamento com pagamentos.
Depois criei o `PagamentoService`. Nele ficam as ações de criar, atualizar e excluir pagamento. Também deixei a função `atualizarStatus`, que chama uma procedure do PostgreSQL.
No CRUD web eu criei o `PagamentoController` e as páginas de lista, cadastro, edição e detalhes. A tela de cadastro usa os alunos que já estão cadastrados no sistema.
Na API eu criei outro controller, o `PagamentoResource` e as rotas em `routes/api.php`. A API faz consulta, cadastro, edição e exclusão.

4 Procedure e trigger

A procedure da minha parte se chama `atualizar_pagamento`. Ela atualiza os pagamentos ativos de um plano. O exemplo de chamada é:

```
CALL atualizar_pagamento('Devendo', 'Anual', 'Pago');
```

A trigger se chama `trg_bloqueio_pagamento`. Ela é executada antes de uma atualização. Se o pagamento estiver inativo, não deixa mudar o status para `Pago`.
A função usada pela trigger é `bloqueia_pagamento`. A regra fica no banco, então não depende só da tela do sistema.

5 Etapas e datas

As datas abaixo são fictícias, como foi combinado para o relatório.

Etapa	O que foi feito	Responsável	Data
1	Leitura do PDF e divisão das tarefas	Grupo	12/10/2026
2	Análise do banco e definição dos campos de pagamento	Juan	14/10/2026
3	Migration, model, request e factory	Juan	16/10/2026
4	Procedure, trigger e `PagamentoService`	Juan	18/10/2026
5	CRUD web de pagamentos	Juan	22/10/2026
6	API, resource e rotas API	Juan	25/10/2026
7	Teste web e teste de API	Juan	27/10/2026
8	Revisão dos arquivos e relatório	Juan	30/10/2026
9	Revisão final do grupo	Grupo	03/11/2026

6 Rotas de pagamentos

6.1 Rotas web

As rotas web usam `Route::resource` dentro do grupo que exige login:

```
Route::resource('pagamentos', PagamentoController::class);
```

Verbo	Rota	Uso
GET	`/pagamentos`	Lista os pagamentos.
GET	`/pagamentos/create`	Abre o cadastro.
POST	`/pagamentos`	Salva um pagamento.
GET	`/pagamentos/{pagamento}`	Mostra um pagamento.
GET	`/pagamentos/{pagamento}/edit`	Abre a edição.
PUT/PATCH	`/pagamentos/{pagamento}`	Atualiza um pagamento.
DELETE	`/pagamentos/{pagamento}`	Exclui um pagamento.

6.2 Rotas da API

As rotas da API usam `Route::apiResource`:

```
Route::apiResource('pagamentos', PagamentoController::class);
```

Verbo	Rota	Uso
GET	`/api/pagamentos`	Lista pagamentos em JSON.
POST	`/api/pagamentos`	Cria um pagamento e retorna 201.
GET	`/api/pagamentos/{pagamento}`	Mostra um pagamento em JSON.
PUT/PATCH	`/api/pagamentos/{pagamento}`	Atualiza um pagamento.
DELETE	`/api/pagamentos/{pagamento}`	Exclui e retorna 204.

7 Testes

Eu criei dois testes em `tests/Feature`.

O `PagamentoRoutesTest` cria um usuário, um aluno e um pagamento. Depois acessa `/pagamentos` e verifica a página e os dados.

O `PagamentoApiTest` cria um aluno e um pagamento. Depois acessa `/api/pagamentos` e verifica o status 200, os campos do JSON e o nome do aluno.

8 Arquivos principais

Os arquivos da minha parte são:

- `app/Models/Pagamento.php`;
- `app/Http/Requests/PagamentoRequest.php`;
- `app/Services/PagamentoService.php`;
- `app/Http/Controllers/PagamentoController.php`;
- `app/Http/Controllers/Api/PagamentoController.php`;
- `app/Http/Resources/PagamentoResource.php`;
- `database/migrations/2026_11_02_000006_create_pagamento_table.php`;
- `database/sql/pagamentos_pessoa3.sql`;
- `tests/Feature/PagamentoRoutesTest.php`;
- `tests/Feature/PagamentoApiTest.php`.

9 Finalização

Eu deixei a minha parte separada em commits pequenos. Primeiro fiz a base do pagamento. Depois fiz a regra do banco e o service. Em seguida fiz o CRUD web. Por último fiz a API, os testes e este relatório.
A execução dos testes precisa ser feita em um ambiente com PHP, Composer e banco configurados. O comando usado será:

```
php artisan test --filter=Pagamento
```